

学号： 1906410229

沈阳建筑大学

上机报告

课程名称： 算法设计与分析

项目名称： 回溯法

2022年秋季学期

上机时间：第 9 周 星期 五 第 3 ~ 4 节

学生姓名： 蔡艺洵 专业班级： 计算机 1902

指导教师： 刘俊岭 成 绩：

项 目 名 称	回溯法	学 时	4
日期： 2022 年 10 月 28 日		第 9 教学周	
硬件及软件			
名 称		版本、型号	
Dev-C++		5.11	
上机目的与要求： 1. 问题描述，题目定义。 2. 算法实例，实例演示程序过程。 3. 程序代码，完整可运行程序，并加注释。 4. 程序验证，要求有不同的输入、输出，并截图。 5. 算法分析，时间复杂度与空间复杂度。			
本次任务： 实验 5-1： 01 背包问题(回溯法) 实验 5-2： 装载问题(回溯法) 实验 5-3： n 皇后问题(回溯法) 实验 5-4： 旅行商问题(回溯法)			

实验 5-1 01 背包问题(回溯法)

1.问题描述:

有 n 个物品，每个物品有重量 w 和价值 v 两个属性，还有一个承重为 W 的背包。现在让你用这个背包装物品，使所装物品的价值最大化。

2.算法实例:

输入: $W = 6$

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

要求:

- 1 填空完成程序；
- 2 输出所选物品重量及价值，总重量与价值。
- *3 报告中指出是排列树还是子集树，指出约束函数与限界函数。

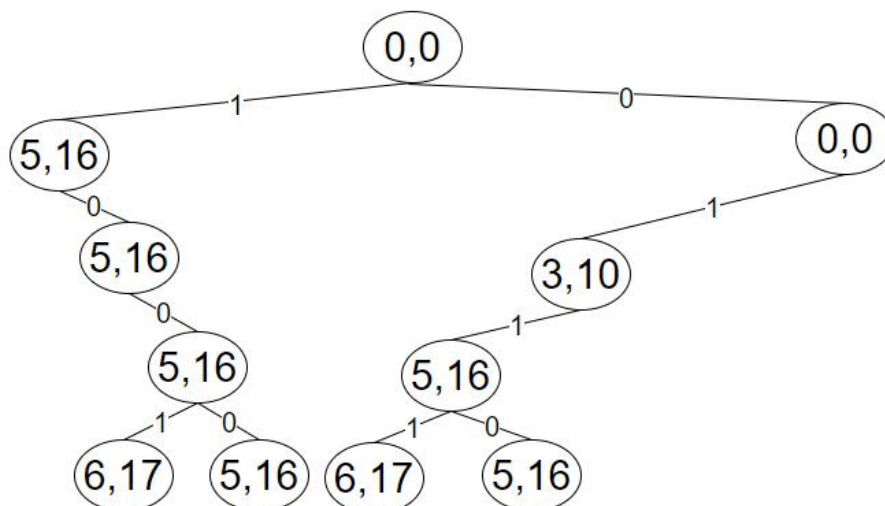
输出:

```

5-1.cpp
10 //问题表示
11 int n = 4;           //4种物品
12 int W = 6;           //限制重量为6
13 //int w[] = { 0, 5, 3, 2, 1 };
14 //int v[] = { 0, 4, 4, 3, 1 };
15 int w[] = { 0, 5, 3, 2, 1 };
16 int v[] = { 0, 16, 10, 6, 1 };
17 //求解结果表示
18 int x[5];            //存放最终解
19 int maxv;            //存放最优解的总
20

```

输入的物品假设为 A、B、C、D，其重量分别为 1 2 3 5，其价值为 1 6 10 16，在代码中用 w 数组记录物品的重量、用 v 数组记录物品的价值，n 为物品的数量、W 为背包的容量。如上图所示。输出结果为选择第一个物品和第四个物品，总重量为 6，总价值为 17。该问题的解空间是子树集，如下图所示，其约束函数与限界函数为 $tw + w[i] \leq W$ 和 $tw + rw > W$ 。



3.程序代码:

```
/*01 背包问题,回溯法。*/
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;
//问题表示
int n = 4;           //4 种物品
int W = 6;           //限制重量为 6
//int w[] = { 0, 5, 3, 2, 1 };      //存放 4 个物品重量,不用下标 0 元素
//int v[] = { 0, 4, 4, 3, 1 };      //存放 4 个物品价值,不用下标 0 元素
int w[] = { 0, 5, 3, 2, 1 };
int v[] = { 0, 16, 10, 6, 1 };
//求解结果表示
int x[5];            //存放最终解
int maxv;            //存放最优解的总价值

void dfs(int i, int tw, int tv, int rw, int op[]); //求解 0/1 背包问题

int main(){
    //假设已按单位价值降序排序
    int sumWeight=0, sumProfit=0;
    int rw=0;
    int op[5] = {0};

    for (int i = 1; i <= n; i++)
    {
        rw+= w[i];
    }
    dfs(1, 0, 0, rw, op);

    printf("放入背包的物品为: \n");
    for (int i = 1; i <= n ; i++)
    {
        if (x[i] != 0){
            printf("第%d 个, 重量为: %d, 价值为: %d\n", i, w[i], v[i]);
            sumWeight += w[i];
            sumProfit += v[i];
        }
        //printf("%f ", x[i]);
    }
    printf("总重量为: %d, 总价值为: %d\n", sumWeight, maxv);
}
```

```

    system("PAUSE");
    return 0;
}

void dfs(int i, int tw, int tv, int rw, int op[]) //求解 0/1 背包问题
{ //初始调用时 rw 为所有物品重量和
    int j;
    if (i>n) //找到一个叶子结点
    {
        if (tw == W && tv>maxv) //找到一个满足条件的更优解,保存
        {
            maxv = tv;
            for (j = 1; j <= n; j++) //复制最优解
                x[j] = op[j];
        }
    }
    else //尚未找完所有物品
    {
        if (tw + w[i] <= W) //左孩子结点剪枝
        {
            op[i] = 1; //选取第 i 个物品
            dfs(i + 1, tw+w[i], tv+v[i], rw+w[i], op);
        }
        op[i] = 0; //不选取第 i 个物品,回溯
        if (tw + rw>W) //右孩子结点剪枝
            dfs(i + 1, tw, tv, rw-w[i], op);
    }
}
}

```

4.程序验证:

```

int n = 4; //4种物品
int W = 6; //限制重量为6
//int w[] = { 0, 5, 3, 2, 1 };
//int v[] = { 0, 4, 4, 3, 1 };
int w[] = { 0, 5, 3, 2, 1 };
int v[] = { 0, 16, 10, 6, 1 };

```

测试 1: 输入的例子为: $w[] = \{0, 5, 3, 2, 1\}$ 、 $v[] = \{0, 16, 10, 6, 1\}$, 在代码中 w 数组是物品的重量数组, 在代码中 v 数组为物品的价值数组, 输出的结果为: 选择物品 1 和物品 4, 总重量为 6, 总价值为 17。

测试 2: 输入的例子为: $w[] = \{0, 5, 3, 2, 1\}$ 、 $v[] = \{0, 4, 4, 3, 1\}$, 在代码中 w 数组是物品的重量数组, 在代码中 v 数组为物品的价值数组, 输出的结果为: 选择物品 2、物品 3 和物品 4, 总重量为 6, 总价值为 8。

该问题是在回溯算法中其解空间是子树集, 回溯算法其约束函数与限界函数为 $tw + w[i] \leq W$ (进行左孩子剪枝) 和 $tw + rw > W$ (进行右孩子剪枝)。结果如图所示:

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第五次上机课\5-1.exe

```
放入背包的物品为：
第1个，重量为：5， 价值为：16
第4个，重量为：1， 价值为：1
总重量为：6， 总价值为：17
请按任意键继续. . .
```

测试 1 的运行截图

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第五次上机课\5-1.exe

```
放入背包的物品为：
第2个，重量为：3， 价值为：4
第3个，重量为：2， 价值为：3
第4个，重量为：1， 价值为：1
总重量为：6， 总价值为：8
请按任意键继续. . .
```

测试 2 的运行截图

5. 算法分析：

该算法使用了回溯算法。在回溯的过程中进行了两次的剪枝，分别是左剪枝 $tw + w[i] \leq W$ 以及右剪枝 $tw + rw > W$ ，对算法的可行性解空间进行了缩小，缩小的最优解的范围，使得搜索更高效。本算法如果不考虑剪枝，其时间复杂度为 $O(2^n)$ ，使用了长度为 n 的一维数组，所以空间复杂度为 n ，

时间复杂度： $O(2^n)$

空间复杂度： $O(n)$

实验 5-2 装载问题(回溯法)

1. 问题描述：

有一批共 n 个集装箱要装上两艘载重量分别为 c_1 和 c_2 的轮船，其中集装箱 i 的重量为 w_i ，且 $w_1 + w_2 + \dots + w_n \leq c_1 + c_2$ 。装载问题要求确定是否有一个合理的装载方案可将这些集装箱装上这两艘轮船。如果有，找出一种装载方案。

2. 算法实例：

当 $n=3$ ， $c_1=c_2=50$ ， $w=\{10, 40, 40\}$ 时，则可以将集装箱 1 和 2 装到第一艘轮船上，而将集装箱 3 装到第二艘轮船上。如果 $w=\{20, 40, 40\}$ ，则无法将这 3 个集装箱都装上轮船。

要求：

1 填空完成程序；

2 输出装载方案。

*3 报告中指出是排列树还是子集树，指出约束函数与限界函数。

输出为：

装载方案为：

将第 1 个集装箱装上第一艘轮船

将第 2 个集装箱装上第一艘轮船

将第 3 个集装箱装上第二艘轮船

其解空间是子集树，约束函数与限界函数是 $tw + w[i] \leq c_1$ （进行左剪枝）和 $tw + rw > c_1$ （进行右剪枝）。

```
求解结果
装载方案
将第1个集装箱装载在第一艘轮船
将第2个集装箱装载在第一艘轮船
将第3个集装箱装载在第二艘轮船
请按任意键继续. . .
```

3.程序代码:

```
/*MaxLoading,回溯法。*/
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>
#include <vector>
#include <algorithm>
#include <cstring>
using namespace std;

//问题表示
int w[] = { 0, 10, 40, 40 }; //各集装箱重量,不用下标 0 的元素
int n = 3;
int c1 = 50, c2 = 50;

//求解结果表示
int maxw; //存放最优解的总重量
int x[6]; //存放最优解向量
int minnum = 999999; //存放最优解的集装箱个数,初值为最大值

void dfs(int tw, int rw, int op[], int i); //求第一艘轮船的最优解
bool solve(); //求解复杂装载问题

int main(){
    //假设已按单位价值降序排序
    int op[6]; //存放临时解

    memset(op, 0, sizeof(op));
    int rw = 0;
    for (int i = 1; i <= n; i++)
        rw += w[i];
    dfs(0, rw, op, 1); //求第一艘轮船的最优解
    printf(" 求解结果\n");
    if (solve()) //输出结果
    {
        printf(" 装载方案\n");
    }
}
```

```

        for (int i = 1; i <= n; i++)
        {
            if(x[i]==1){
                printf("将第%d 个集装箱装载在第一艘轮船\n ",i);
            }
            else{
                printf("将第%d 个集装箱装载在第二艘轮船\n",i);
            }
            //printf("%d ",x[i]);
        }

    }
    else printf("    没有合适的装载方案\n");

    system("PAUSE");
    return 0;
}

void dfs(int tw, int rw, int op[], int i) //求第一艘轮船的最优解
{
    if (i>n)                //找到一个叶子结点
    {
        if (tw <= c1 && tw>maxw)
        {
            maxw =tw;        //找到一个满足条件的更优解
            for (int j = 1; j <= n; j++)    //复制最优解
                x[j] =op[j];
        }
    }
    else                    //尚未找完所有集装箱
    {
        op[i] = 1;        //选取第 i 个集装箱
        if (tw + w[i] <= c1) //左孩子结点剪枝
            dfs(tw+w[i],rw-w[i], op, i + 1);
        op[i] = 0;        //不选取第 i 个集装箱,回溯
        if (tw + rw>c1)    //右孩子结点剪枝
            dfs(tw,rw-w[i], op, i + 1);
    }
}

bool solve()                //求解复杂装载问题
{
    int sum = 0;            //累计第一艘轮船装完后剩余的集装箱重量
    for (int j = 1; j <= n; j++)
        if (x[j] == 0)
            sum += w[j];
    if (sum <= c2)        //第二艘轮船可以装完

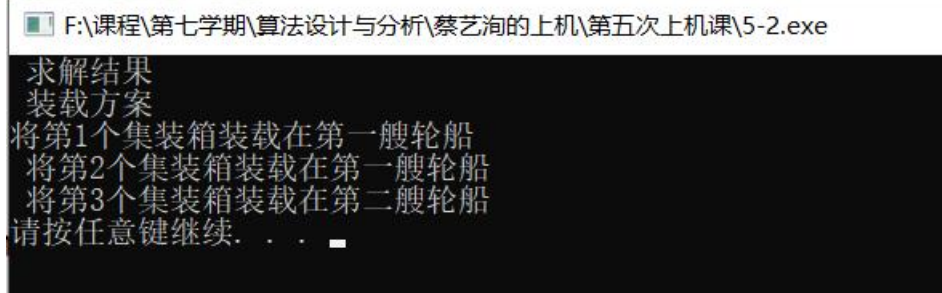
```



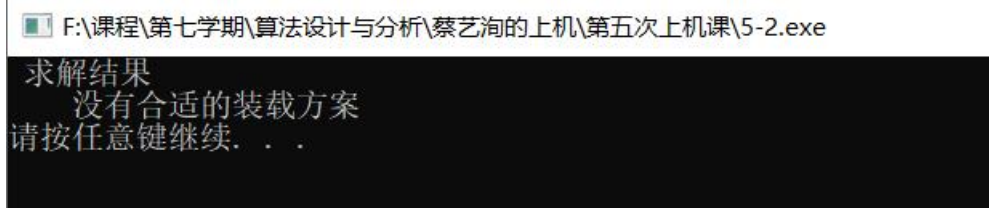
```
        return true;
    else        //第二艘轮船不能装完
        return false;
}
```

4.程序验证:

验证实例的结果，输入的数据为 $n=3$ ，即有三个集装箱，俩艘船，分别是 c_1 ， c_2 ，装载量均为 50，三个集装箱的重量分别是 10，40，40。运行程序得到以下截图：



但当将上面的例子的数据进行修改，其他数据不变，将三个集装箱的重量改为 20，40，40。运行结果如下图：



程序验证正确，可以得到问题的装载方案，如没有合适的装载方案，会返回如上图的结果。该问题使用了回溯算法进行求解，回溯的树是子集树，其约束函数与限界函数是 $tw + w[i] \leq c_1$ （进行左剪枝）和 $tw + rw > c_1$ （进行右剪枝）。

5.算法分析:

本算法求解的问题是装载问题，每次进行选择是否装载，所以是子集树，所以其时间复杂度为 $O(2^n)$ 。如果对子集树进行剪枝，则用 $tw + w[i] \leq c_1$ 进行左孩子剪枝，用 $tw + rw > c_1$ 进行右孩子剪枝，这样可以优化算法的时间复杂度，而空间复杂度，本次算法只用了 2 个长度为 n 的一维数组，所以空间复杂度为 $O(n)$ 。所以

- 时间复杂度： $O(2^n)$
- 空间复杂度： $O(n)$

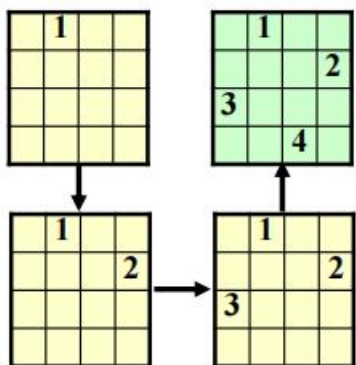
实验 5-3 n 皇后问题(回溯法)

1.问题描述:

在 $n \times n$ 的棋盘上摆放 n 个皇后，使其不能互相攻击，即任意两个皇后都不能处于同一行、同一列或同一斜线上。

2.算法实例:

在 4×4 的棋盘上摆放四个皇后，使其不能互相攻击，即任意两个皇后都不能处于同一行、同一列或同一斜线上。



输出:

要求:

要求:

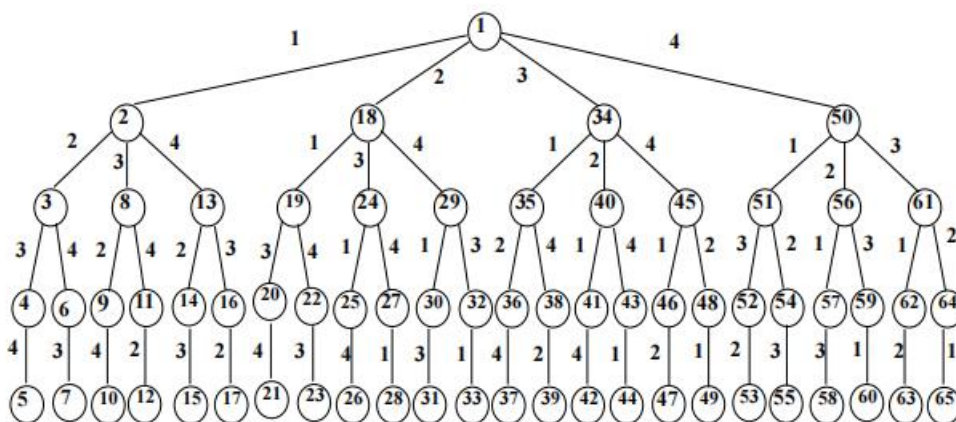
1 填空完成程序;

2 输出每个解 n 个皇后的位置，给出解的个数。

*3 分析解空间的分支总数，利用程序计算回溯法生成的分支数。

*4 报告中指出是排列树还是子集树。

4皇后问题解空间的树结构



四皇后的解空间树结构如上图所示。该解空间是排列树，在未进行剪枝的情况下，分支总数为 64，最后得到两个解，其表示方法按照 1、2、3、4 行默认顺序进行表示，如下：

解序号 0: 2 4 1 3 完整表示为: (1, 2) (2, 4) (3, 1) (4, 3)

解序号 1: 3 1 4 2 完整表示为: (1, 3) (2, 1) (3, 4) (4, 2)

3.程序代码:

```
/*nQueen 回溯法,排列树。*/
#define _CRT_SECURE_NO_WARNINGS

#include <iostream>
#include <vector>
#include <string>
using namespace std;

void Queens(vector<int>& q, int i, int length);
bool place(vector<int>& q, int i);
void swap(vector<int>& q, int i, int j);
int cnt = 0; //解的数量
int sum=0;
int main()
{
    int n = 4;
    vector<int> q(n); //存放最优解向量
    for (int i = 0; i < n; i++)
    {
        q[i] = i + 1; //下标表示行/列, 值表示列/行
    }
    Queens(q, 0, n);
    cout << "共有" << cnt << "个解" << endl;
    cout << "共有" << sum << "个分支" << endl;
    system("PAUSE");
    return 0;
}

void Queens(vector<int>& q, int i, int length)
{
    if (i == length) //找到一个解
    {
        cout << "解序号" << cnt++ << ": ";
        for (int j = 0; j < length; j++)
        {
            cout << q[j] << " ";
        }
        cout << endl;
    }
    else
    {
        for (int k = i; k < length; k++)
        {
            swap(q, k, i);
```

```

        //sum++;
        if (place(q, i)) //是否可放置
            Queens(q, i+1, length);
        swap(q, k, i); //回溯还原
    }
}
//测试第 i 行的 q[i]列上能否摆放皇后
bool place(vector<int>& q, int i)
{
    sum++;
    for (int j = 0; j < i; j++)
    {
        if (i == j || q[i] == q[j] || abs(i-j) == abs(q[i] - q[j])) //同行或同列或在同一斜线上
        {
            return false;
        }
    }
    return true;
}

void swap(vector<int>& q, int i, int j)
{
    int tmp = q[i];
    q[i] = q[j];
    q[j] = tmp;
}

```

4.程序验证:

在输入例子为四皇后的例子，原本的解空间有 64 个分支，对其进行剪枝，使得解空间中存在的分支都为符合约束条件下的分支，对分支进行计数，结果如图所示：

 F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第五次上机课\5-3.exe

```

解序号0: 2 4 1 3
解序号1: 3 1 4 2
共有2个解
共有32个分支
请按任意键继续. . .

```

测试用例改为八皇后的问题，将代码中的 n 的值改为 8，我们可以得到 92 种解，分别以默认的行顺序 1、2、3、4、5、6、7、8 进行表示，解序号从 0 开始，例如解序号 0: 1 5 8 6 3 7 2 4 表示的是解法的皇后落子点为 (1, 1) (2, 5) (3, 8) (4, 6) (5, 3) (6, 7) (7, 2) (8, 4)，其他解序号的表示也是如此，运行结果共有 92 种解法，其剪枝之后的分支总数为 5508 个分支。该算法下的解空间为排列树，对排列树进行了优化剪枝。

运行结果如下：

```

解序号65: 6 2 7 1 3 5 8 4
解序号66: 6 3 5 7 1 4 2 8
解序号67: 6 3 5 8 1 4 2 7
解序号68: 6 3 1 7 5 8 2 4
解序号69: 6 3 1 8 5 2 4 7
解序号70: 6 3 1 8 4 2 7 5
解序号71: 6 3 7 4 1 8 2 5
解序号72: 6 3 7 2 4 8 1 5
解序号73: 6 3 7 2 8 5 1 4
解序号74: 6 4 2 8 5 7 1 3
解序号75: 6 4 1 5 8 2 7 3
解序号76: 6 4 7 1 3 5 2 8
解序号77: 6 4 7 1 8 2 5 3
解序号78: 6 1 5 2 8 3 7 4
解序号79: 6 8 2 4 1 7 5 3
解序号80: 7 2 4 1 8 5 3 6
解序号81: 7 2 6 3 1 4 8 5
解序号82: 7 3 1 6 8 5 2 4
解序号83: 7 3 8 2 5 1 6 4
解序号84: 7 4 2 5 8 1 3 6
解序号85: 7 4 2 8 6 1 3 5
解序号86: 7 5 3 1 6 8 2 4
解序号87: 7 1 3 8 6 4 2 5
解序号88: 8 2 4 1 7 5 3 6
解序号89: 8 2 5 3 1 7 4 6
解序号90: 8 3 1 6 2 5 7 4
解序号91: 8 4 1 3 6 2 7 5
共有92个解
共有5508个分支
请按任意键继续. . .

```

5. 算法分析:

本算法的解空间为排列树，在未进行剪枝的情况下，时间复杂度为 $O(n!)$ ，在加入限制条件下对解空间进行优化剪枝，时间复杂度会有一定的优化。空间开销只有一个长度为 n 的一维数组，用来记录最优解。 n 代表求解的问题为 n 皇后问题，所以空间复杂度为 $O(n)$ 。

时间复杂度为 $O(n!)$

空间复杂度为 $O(n)$

实验 5-4 旅行商问题(回溯法)

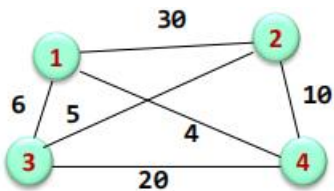
1. 问题描述:

某售货员要到 n 个城市去推销商品，已知各城市之间的路程，请问他应该如何选定一条从城市 1 出发，经过每个城市一遍，最后回到城市 1 的路线，使得总的周游路程最小。

2. 算法实例:

示例 1: 结点数 4

输入: [0 30 6 4][30 0 5 10][6 5 0 20][4 10 20 0] 用图表示如下:



输出:

要求:

- 1 填空完成程序;
- 2 输出最少旅行费用, 给出旅行路径。
- *3 分析解空间的分支总数, 利用程序计算回溯法生成的分支数。
- *4 报告中指出是排列树还是子集树, 指出约束函数与限界函数

```

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第五次上机课\5-4.exe
输入城市个数:
4
输入城市之间的距离(0表示城市间不通):
0 30 6 4
30 0 5 10
6 5 0 20
4 10 20 0
最少旅行费用为: 25
旅行路径为:
1 3 2 4 1请按任意键继续. . .
  
```

3.程序代码:

```

/**
回溯法-旅行商(TSP)问题
输入
4
0 30 6 4
30 0 5 10
6 5 0 20
4 10 20 0
*/

#define _CRT_SECURE_NO_WARNINGS
#include<iostream>
#include<algorithm>
#define MAX 100
using namespace std;

//问题表示
int sum=0;
int n; //城市个数
int a[MAX][MAX]; //城市间距离,邻接矩阵, 0 表示不通
//求解结果表示
int x[MAX]; //记录路径
  
```

```

int bestx[MAX] = { 0 };           //记录最优路径
int bestp = 63355;                //最短路径长
int cp = 0;                       //当前路径长

void backtrack(int t){
    if (t>n){//得到一个解
        sum++;
        if ((a[x[n]][1]) && (a[x[n]][1] + cp<bestp)){
            bestp =a[x[n]][1]+cp;
            for (int i = 1; i <= n; i++){
                bestx[i] =x[i];
            }
        }
    }
    else{
        for (int i = t; i <= n; i++){
            sum++;
            /*约束为当前节点到下一节点的长度不为 0,限界为走过的长度+当前要走的长度之和小于最
            优长度*/
            if ((a[x[t - 1]][x[i]]) && (cp + a[x[t - 1]][x[i]]<bestp)){
                swap(x[t], x[i]);
                cp +=a[x[t-1]][x[t]];
                backtrack(t + 1);
                cp -= a[x[t-1]][x[t]];
                swap(x[t], x[i]);
            }
        }
    }
}

int main(){
    cout << "输入城市个数:" << endl;
    cin >> n;           //顶点数
    for (int i = 1; i <= n; i++){
        x[i] = i;
    }
    cout << "输入城市之间的距离(0 表示城市间不通):" << endl;
    for (int i = 1; i <= n; i++){
        for (int j = 1; j <= n; j++){
            cin >> a[i][j];
        }
    }
    backtrack(2);
    cout << "最少旅行费用为:" << bestp << endl;
    cout << "旅行路径为:" << endl;
    for (int i = 1; i <= n; i++){
        cout << bestx[i] << " ";
    }
}

```



```

cout << bestx[1]<<endl;
cout << "解空间下的结点总数为:" <<sum--<< endl;
cout << "解空间下的分支总数为:" <<sum<< endl;
system("PAUSE");
return 0;
}

```

4.程序验证:

验证例子 1: 输入的城市数为 4, 其彼此直接的情况为[0 30 6 4][30 0 5 10][6 5 0 20][4 10 20 0],此为城市间距离,邻接矩阵, 0 表示不通, 如图所示:

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第五次上机课\5-4.exe

```

输入城市个数:
4
输入城市之间的距离(0表示城市间不通):
0 30 6 4
30 0 5 10
6 5 0 20
4 10 20 0

```

输出的结果为: 最少旅行费用为 25, 旅行路径为: 1 3 2 4 1 结果如图所示:

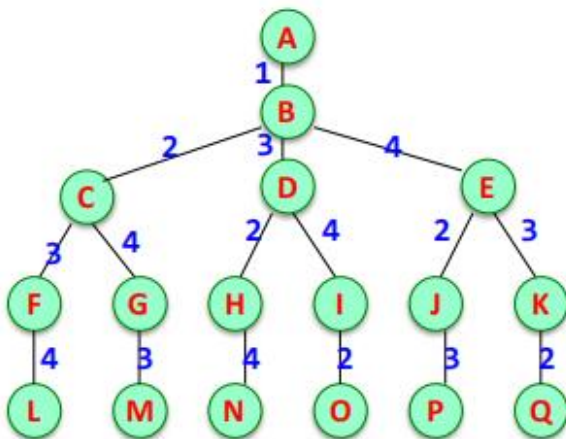
F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第五次上机课\5-4.exe

```

输入城市个数:
4
输入城市之间的距离(0表示城市间不通):
0 30 6 4
30 0 5 10
6 5 0 20
4 10 20 0
最少旅行费用为: 25
旅行路径为:
1 3 2 4 1
请按任意键继续. . .

```

该问题是一个 NP 完全问题, 有 $(n-1)!$ 条可选路线:



该问题解空间为用图的邻接矩阵 a 表示无向连通图 $G = (V, E)$ 。若 (i, j) 属于图的边集, 则 $a[i][j]$ 表示两个城市之间的距离, 若 $a[i][j] = 0$, 表示城市之间无通路。问题的解空间就是出发城市+其他城市全排列。

本问题的解空间结构为一颗排列树。当结点数为 4 时，其分支总数为 16，节点数为 17，其中对其进行剪枝，约束函数与限界函数为 $(a[x[t - 1]][x[i]]) \ \&\& \ (cp + a[x[t - 1]][x[i]] < bestp)$ ，表示当前节点到下一节点的长度不为 0,限界为走过的长度+当前要走的长度之和小于最优长度，运行如下图：

 F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第五次上机课\5-4.exe

```
输入城市个数:
4
输入城市之间的距离(0表示城市间不通):
0 30 6 4
30 0 5 10
6 5 0 20
4 10 20 0
最少旅行费用为: 25
旅行路径为:
1 3 2 4 1
解空间下的结点总数为:17
解空间下的分支总数为:16
请按任意键继续. . .
```

5.算法分析:

本算法使用了回溯法来对城市进行选择，测试的例子中，城市数为 4，给出相对应的距离矩阵，本算法的解空间为排列树，其空间复杂度为 $O(n!)$

如果使用约束函数与限界函数 $(a[x[t - 1]][x[i]]) \ \&\& \ (cp + a[x[t - 1]][x[i]] < bestp)$ 进行剪枝，则时间开销会小一点。

另外，代码中使用了几个一维的数组来记录路径、最优路径等数据，使用一个二维的 n 阶方阵来记录与表示城市之间的距离。所以空间开销为 $O(n*n)$ 。

时间复杂度: $O(n!)$

空间复杂度: $O(n*n)$

分析与总结：

回溯法，又称为试探法。类似穷举的搜索尝试过程，主要是在搜索尝试过程中寻找问题的解，当发现已不满足求解条件时，就“回溯”（即回退），尝试别的路径。回溯法普遍采用的是深度优先搜索算法，即DFS。其解空间可分为子集树与排列树两种结构，一般来说，子集树的时间复杂度为 $O(2^n)$ ，排列树的时间复杂度为 $O(n!)$ ，但因为实际问题的不同，时间复杂度会有不同的变化，例如 n 皇后问题，如果采用子集树，则时间复杂度为 $O(n^n)$ ，并不是 $O(2^n)$ ，所以实际问题要采用子集树还是排列树，需要具体分析。这是比较麻烦的一点。回溯法搜索解空间时，通常采用两种策略避免无效搜索，提高回溯的搜索效率：1. 用约束函数在扩展结点处剪除不满足约束的子树；2. 用限界函数剪去得不到问题解或最优解的子树。这两类函数统称为剪枝函数。实际上，回溯法基本上会伴随着剪枝，这也是回溯法常见的算法优化方向。这次的四个实验题，都是回溯算法，也都使用了剪枝函数，可见回溯与剪枝是一对孪生兄弟。

用回溯法解题的一般步骤如下：

- ① 针对所给问题，确定问题的解空间树，问题的解空间树应至少包含问题的一个（最优）解。
- ② 确定结点的扩展搜索规则。
- ③ 以深度优先方式搜索解空间树，并在搜索过程中可以采用剪枝函数来避免无效搜索

本 次 上 机 成 绩	项目	操作	报告书写	出勤和课堂纪律	其他
	得分				
	成绩合计：				

评语：

教师签字：

日期：